

LAPORAN PRAKTIKUM ALSRUDAT

Selection Sort, Bubble Sort, Insertion Sort



Nama : Yohansen Hutahaeon

NIM : 42325034

Prodi : D3 Teknologi Informasi

1. Selection Sort

```
[1]: def selection_sort(arr):
    """
    Mengimplementasikan algoritma Selection Sort
    """
    n = len(arr) # Mendapatkan jumlah elemen dalam array

    # Melakukan perulangan untuk setiap elemen di dalam array
    for i in range(n):
        # Mengasumsikan elemen saat ini (i) adalah yang terkecil (minimum)
        min_idx = i

        # Perulangan untuk mencari elemen terkecil di sisa array yang belum terurut
        for j in range(i + 1, n):
            # Jika menemukan elemen yang lebih kecil dari elemen di min_idx
            if arr[j] < arr[min_idx]:
                min_idx = j # Perbarui indeks elemen terkecil

        # Tukar elemen terkecil yang ditemukan dengan elemen pertama di bagian belum terurut
        arr[i], arr[min_idx] = arr[min_idx], arr[i]

    return arr

# Contoh penggunaan
data = [64, 25, 12, 22, 11]
print("Array sebelum diurutkan:", data)
sorted_data = selection_sort(data.copy())
print("Array setelah Selection Sort:", sorted_data)

Array sebelum diurutkan: [64, 25, 12, 22, 11]
Array setelah Selection Sort: [11, 12, 22, 25, 64]
```

Penjelasan :

1. `def selection_sort(arr):`: Mendefinisikan fungsi bernama `selection_sort` yang menerima satu parameter berupa list/array `arr`.
2. `n = len(arr)`: Menghitung panjang array dan menyimpannya dalam variabel `n`.
3. `for i in range(n)`: Perulangan luar untuk melewati setiap posisi dalam array.
4. `min_idx = i`: Menandai indeks saat ini sebagai lokasi nilai terkecil sementara.
5. `for j in range(i + 1, n)`: Perulangan dalam untuk membandingkan nilai di `min_idx` dengan elemen-elemen setelahnya.
6. `if arr[j] < arr[min_idx]`: Mengecek apakah elemen di posisi `j` lebih kecil dari elemen di posisi `min_idx`.
7. `min_idx = j`: Jika ya, simpan indeks `j` tersebut sebagai indeks terkecil yang baru.
8. `arr[i], arr[min_idx] = arr[min_idx], arr[i]`: Menukar posisi nilai di indeks `i` dengan nilai di indeks terkecil yang ditemukan.
9. `return arr`: Mengembalikan array yang sudah dalam kondisi terurut.

2. Bubble Sort

```
[2]: def bubble_sort(arr):  
    """  
    Mengimplementasikan algoritma Bubble Sort  
    """  
    n = len(arr)  
  
    # Melakukan perulangan melalui semua elemen array  
    for i in range(n):  
        # Variabel untuk menandai apakah ada pertukaran yang terjadi  
        swapped = False  
  
        # Perulangan untuk membandingkan elemen yang bersebelahan  
        # Elemen terakhir ke-i sudah pasti berada di posisi yang benar  
        for j in range(0, n - i - 1):  
            # Jika elemen saat ini lebih besar dari elemen berikutnya  
            if arr[j] > arr[j + 1]:  
                # Tukar kedua elemen tersebut  
                arr[j], arr[j + 1] = arr[j + 1], arr[j]  
                swapped = True # Tandai bahwa terjadi pertukaran  
  
        # Jika tidak ada elemen yang ditukar di perulangan dalam, berarti array sudah urut  
        if not swapped:  
            break  
  
    return arr  
  
# Contoh penggunaan  
data = [64, 34, 25, 12, 22, 11, 90]  
print("Array sebelum diurutkan:", data)  
sorted_data = bubble_sort(data.copy())  
print("Array setelah Bubble Sort:", sorted_data)  
  
Array sebelum diurutkan: [64, 34, 25, 12, 22, 11, 90]  
Array setelah Bubble Sort: [11, 12, 22, 25, 34, 64, 90]
```

Penjelasan :

1. `for i in range(n)::` Perulangan luar yang menentukan berapa kali kita akan menyisir array.
2. `swapped = False:` Inisialisasi variabel untuk optimasi. Jika dalam satu putaran tidak ada yang ditukar, proses berhenti.
3. `for j in range(0, n - i - 1)::` Perulangan dalam untuk membandingkan tetangga. Batas `n-i-1` digunakan karena elemen terbesar akan "mengapung" ke ujung belakang setiap kali putaran selesai.
4. `if arr[j] > arr[j + 1]:` Mengecek apakah elemen di kiri lebih besar dari elemen di kanan.
5. `arr[j], arr[j + 1] = arr[j + 1], arr[j]:` Menukar posisi jika kondisi di atas terpenuhi.
6. `swapped = True:` Mencatat bahwa ada perubahan, sehingga perulangan luar perlu berlanjut.
7. `if not swapped: break:` Jika sampai akhir perulangan dalam tidak ada yang ditukar, maka array sudah urut dan fungsi keluar dari loop (optimasi).

3. Insertion Sort

```
[3]: def insertion_sort(arr):  
    """  
    Mengimplementasikan algoritma Insertion Sort  
    """  
    # Mulai dari elemen kedua (indeks 1) hingga elemen terakhir  
    for i in range(1, len(arr)):  
        key = arr[i] # Elemen yang akan disisipkan ke posisi yang benar  
  
        # Memindahkan elemen-elemen dari arr[0..i-1] yang lebih besar dari key  
        # ke satu posisi di depan posisi mereka saat ini  
        j = i - 1  
        while j >= 0 and key < arr[j]:  
            arr[j + 1] = arr[j] # Geser elemen ke kanan  
            j -= 1  
  
        # Letakkan 'key' di posisi yang tepat  
        arr[j + 1] = key  
  
    return arr  
  
    # Contoh penggunaan  
    data = [12, 11, 13, 5, 6]  
    print("Array sebelum diurutkan:", data)  
    sorted_data = insertion_sort(data.copy())  
    print("Array setelah Insertion Sort:", sorted_data)  
  
Array sebelum diurutkan: [12, 11, 13, 5, 6]  
Array setelah Insertion Sort: [5, 6, 11, 12, 13]
```

Penjelasan :

1. `for i in range(1, len(arr)):`: Perulangan dimulai dari indeks 1 karena elemen pertama dianggap sudah "terurut" di bagian kiri.
2. `key = arr[i]`: Menyimpan nilai elemen yang sedang diproses ke dalam variabel `key`.
3. `j = i - 1`: Menetapkan variabel `j` sebagai indeks elemen tepat di sebelah kiri `key`.
4. `while j >= 0 and key < arr[j]:`: Selama indeks `j` masih dalam jangkauan dan nilai di kiri lebih besar dari `key`.
5. `arr[j + 1] = arr[j]`: Geser elemen yang lebih besar tersebut satu posisi ke kanan.
6. `j -= 1`: Pindah ke elemen sebelumnya di sebelah kiri untuk pengecekan berikutnya.
7. `arr[j + 1] = key`: Setelah menemukan posisi yang tepat (di mana tidak ada lagi nilai yang lebih besar di kiri), masukkan `key` ke posisi tersebut.

4. Perbandingan Waktu Eksekusi

```
[4]: import time
import random

def measure_sorting_time(sort_function, data):
    """
    Mengukur waktu eksekusi algoritma pengurutan
    """
    data_copy = data.copy() # Selalu salin data agar tidak merusak data asli
    start_time = time.time() # Catat waktu mulai
    sort_function(data_copy) # Jalankan fungsi pengurutan
    end_time = time.time() # Catat waktu selesai
    return end_time - start_time # Hitung selisih waktu

# Menghasilkan data acak untuk pengujian
data_sizes = [100, 500, 1000]
results = {}

for size in data_sizes:
    # Membuat list angka acak antara 1 s/d 1000 sebanyak 'size'
    random_data = [random.randint(1, 1000) for _ in range(size)]

    # Ukur waktu masing-masing algoritma
    selection_time = measure_sorting_time(selection_sort, random_data)
    bubble_time = measure_sorting_time(bubble_sort, random_data)
    insertion_time = measure_sorting_time(insertion_sort, random_data)

    # Simpan hasil dalam dictionary
    results[size] = {
        'Selection Sort': selection_time,
        'Bubble Sort': bubble_time,
        'Insertion Sort': insertion_time
    }

# Menampilkan hasil dalam bentuk tabel sederhana
print("Perbandingan Waktu Eksekusi (detik):")
print("Size\tSelection Sort\tBubble Sort\tInsertion Sort")
for size, times in results.items():
    print(f"{size}\t{times['Selection Sort']:.6f}\t{times['Bubble Sort']:.6f}\t{times['Insertion Sort']:.6f}")
```

Size	Selection Sort	Bubble Sort	Insertion Sort
100	0.015979	0.001587	0.000550
500	0.110061	0.069849	0.022625
1000	0.151525	0.131702	0.069541

Penjelasan :

1. `import time, random`: Memasukkan pustaka untuk menghitung waktu dan membuat angka acak.
2. `measure_sorting_time(...)`: Fungsi pembantu yang mengambil fungsi urut dan data sebagai input.
3. `data_copy = data.copy()`: Sangat penting untuk menyalin data agar setiap algoritma mengurutkan kumpulan angka yang identik dan belum terurut.
4. `start_time = time.time()`: Mengambil timestamp saat ini.
5. `end_time - start_time`: Menghasilkan durasi proses pengurutan dalam satuan detik.
6. `data_sizes = [100, 500, 1000]`: Menentukan tiga skenario jumlah data untuk melihat perbandingan performa saat beban bertambah.
7. `random.randint(1, 1000)`: Menghasilkan angka acak agar tes bersifat objektif.
8. `f'{size}\t{times['Selection Sort']:.6f}\t{times['Bubble Sort']:.6f}\t{times['Insertion Sort']:.6f}'`: Mencetak hasil dengan format 6 angka di belakang koma agar perbedaan waktu yang sangat kecil bisa terlihat.

5. Implementasi Fitur Tambahan

```
print("=== HASIL FITUR TAMBAHAN ===")

# 1. Menguji Selection Sort Descending (Besar ke Kecil)
data_desc = [11, 25, 12, 22, 64]
print("\n[Selection Sort Descending]")
print("Data awal      :", data_desc)
hasil_desc = selection_sort_desc(data_desc.copy())
print("Hasil (Menurun):", hasil_desc)

# 2. Menguji Bubble Sort dengan Penghitung Pertukaran (Swap)
data_swap = [64, 34, 25, 12, 22, 11, 90]
print("\n[Bubble Sort with Counter]")
print("Data awal      :", data_swap)

# Karena fungsi ini mengembalikan dua nilai (array dan jumlah swap)
hasil_urut, jumlah_swap = bubble_sort_with_counter(data_swap.copy())

print("Hasil Terurut      :", hasil_urut)
print("Jumlah Pertukaran/Swap:", jumlah_swap)

=== HASIL FITUR TAMBAHAN ===

[Selection Sort Descending]
Data awal      : [11, 25, 12, 22, 64]
Hasil (Menurun): [64, 25, 22, 12, 11]

[Bubble Sort with Counter]
Data awal      : [64, 34, 25, 12, 22, 11, 90]
Hasil Terurut   : [11, 12, 22, 25, 34, 64, 90]
Jumlah Pertukaran/Swap: 14
```

Penjelasan :

A. Selection Sort Descending (Besar ke Kecil)

- `def selection_sort_desc(arr):`: Fungsi untuk mengurutkan dari nilai terbesar ke terkecil.
- `max_idx = i`: Mengasumsikan posisi saat ini untuk diisi nilai terbesar.
- `if arr[j] > arr[max_idx]`: Mencari apakah ada angka yang lebih besar di sisa array. Jika ada, `max_idx` diperbarui. Logika `>` inilah yang mengubah urutan menjadi menurun.

B. Bubble Sort with Counter (Menghitung Pertukaran)

- `swap_count = 0`: Wadah untuk menghitung berapa kali terjadi aksi tukar posisi.
- `swap_count += 1`: Setiap kali kode pertukaran (`arr[j], arr[j+1] = ...`) jalan, nilai ini bertambah.
- `return arr, swap_count`: Mengembalikan dua hasil sekaligus: datanya dan jumlah total pertukarannya.

6. Program Perbandingan Performa

```
[6]: def benchmark_sorting():
    size = 1000
    # Persiapan 3 Jenis Data
    data_terurut = list(range(size))
    data_terbalik = list(range(size, 0, -1))
    data_acak = [random.randint(1, 1000) for _ in range(size)]

    datasets = {
        "Data Terurut": data_terurut,
        "Data Terurut Terbalik": data_terbalik,
        "Data Acak": data_acak
    }

    algorithms = {
        "Selection Sort": selection_sort,
        "Bubble Sort": bubble_sort,
        "Insertion Sort": insertion_sort
    }

    print(f"Perbandingan Performa (Data Size: {size})")
    print("-" * 65)
    print(f"{'Kondisi Data':<25} | {'Selection':<10} | {'Bubble':<10} | {'Insertion':<10}")
    print("-" * 65)

    for label, data in datasets.items():
        row = f"{'label':<25} | "
        for name, func in algorithms.items():
            start = time.time()
            func(data.copy())
            end = time.time()
            row += f"{'end - start':<.5f}s | "
        print(row)

benchmark_sorting()
```

```
Perbandingan Performa (Data Size: 1000)
-----
Kondisi Data      | Selection | Bubble   | Insertion
-----
Data Terurut      | 0.05908s  | 0.00016s | 0.00025s |
Data Terurut Terbalik | 0.09346s  | 0.19039s | 0.12035s |
Data Acak         | 0.05253s  | 0.21170s | 0.09760s |
```

Penjelasan :

1. `def benchmark_sorting():`: Mendefinisikan fungsi utama untuk melakukan pengujian performa.
2. `size = 1000`: Menentukan jumlah elemen data yang akan diurutkan (1000 angka).
3. `data_terurut = list(range(size))`: Membuat list angka yang sudah urut dari 0 sampai 999.
4. `data_terbalik = list(range(size, 0, -1))`: Membuat list angka urut terbalik dari 1000 sampai 1.
5. `data_acak = [random.randint(1, 1000) for _ in range(size)]`: Membuat list berisi 1000 angka acak antara 1 sampai 1000.

6. `datasets = { ... }`: Menyimpan ketiga jenis data tersebut ke dalam sebuah *dictionary* untuk memudahkan perulangan.
7. `algorithms = { ... }`: Menyimpan nama dan fungsi algoritma yang akan diuji ke dalam *dictionary*.
8. `print(...)`: Baris-baris ini digunakan untuk mencetak judul tabel dan kepala tabel agar rapi (menggunakan format `:<25` untuk mengatur lebar kolom).
9. `for label, data in datasets.items():`: Perulangan luar untuk melewati setiap jenis kondisi data (Terurut, Terbalik, Acak).
10. `for name, func in algorithms.items():`: Perulangan dalam untuk menguji setiap algoritma pada data yang sedang dipilih.
11. `start = time.time()`: Mencatat waktu tepat sebelum algoritma mulai bekerja.
12. `func(data.copy())`: Menjalankan algoritma pengurutan.
Menggunakan `.copy()` sangat krusial agar data asli tidak berubah saat diuji oleh algoritma berikutnya.
13. `end = time.time()`: Mencatat waktu tepat setelah algoritma selesai bekerja.
14. `row += f'{end - start:.5f}s | '`: Menghitung selisih waktu dan menambahkannya ke string baris tabel dengan format 5 angka di belakang koma.
15. `benchmark_sorting()`: Memanggil fungsi tersebut untuk menjalankan seluruh rangkaian pengujian.

7. Visualisasi Sederhana Proses Pengurutan

```
[8]: def visualize_sorts():
    sample = [5, 2, 9, 1, 5]

    print("\nVisualisasi Selection Sort (Ascending):")
    arr = sample.copy()
    print(f"Awal: {arr}")
    for i in range(len(arr)):
        min_idx = i
        for j in range(i+1, len(arr)):
            if arr[j] < arr[min_idx]: min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
        print(f"Step {i+1}: {arr}")

    print("\nVisualisasi Bubble Sort (Step by Swap):")
    arr = sample.copy()
    print(f"Awal: {arr}")
    step = 1
    for i in range(len(arr)):
        for j in range(0, len(arr)-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
                print(f"Swap {step}: {arr}")
                step += 1

    print("\nVisualisasi Insertion Sort:")
    arr = sample.copy()
    print(f"Awal: {arr}")
    for i in range(1, len(arr)):
        key = arr[i]
        j = i-1
        while j >= 0 and key < arr[j]:
            arr[j+1] = arr[j]
            j -= 1
        arr[j+1] = key
        print(f"Step {i}: {arr}")

visualize_sorts()
```

Visualisasi Selection Sort (Ascending):

Awal: [5, 2, 9, 1, 5]
Step 1: [1, 2, 9, 5, 5]
Step 2: [1, 2, 9, 5, 5]
Step 3: [1, 2, 5, 9, 5]
Step 4: [1, 2, 5, 5, 9]
Step 5: [1, 2, 5, 5, 9]

Visualisasi Bubble Sort (Step by Swap):

Awal: [5, 2, 9, 1, 5]
Swap 1: [2, 5, 9, 1, 5]
Swap 2: [2, 5, 1, 9, 5]
Swap 3: [2, 5, 1, 5, 9]
Swap 4: [2, 1, 5, 5, 9]
Swap 5: [1, 2, 5, 5, 9]

Visualisasi Insertion Sort:

Awal: [5, 2, 9, 1, 5]
Step 1: [2, 5, 9, 1, 5]
Step 2: [2, 5, 9, 1, 5]
Step 3: [1, 2, 5, 9, 5]

Penjelasan :

1. def visualize_sorts():: Mendefinisikan fungsi untuk menjalankan visualisasi pengurutan.

2. `sample = [5, 2, 9, 1, 5]`: Menentukan data contoh yang kecil agar langkah-langkahnya mudah diikuti dan tidak terlalu panjang.
3. `arr = sample.copy()`: Menyalin data contoh ke variabel `arr` agar data asli `sample` tetap tersedia untuk pengujian algoritma berikutnya.
4. `print(f"Awal: {arr}")`: Menampilkan kondisi angka-angka di dalam array sebelum mulai diurutkan.

Pada Bagian Selection Sort:

- `print(f"Step {i+1}: {arr}")`: Perintah cetak diletakkan setelah satu kali proses pencarian nilai minimum dan penukaran selesai. Ini menunjukkan hasil akhir dari setiap putaran (scan) array.

Pada Bagian Bubble Sort:

- `step = 1`: Membuat variabel penghitung untuk menandai urutan setiap terjadi pertukaran.
- `print(f"Swap {step}: {arr}")`: Perintah cetak diletakkan tepat di dalam blok if pertukaran. Ini menunjukkan setiap kali ada angka yang "bergeser/bertukar" tempat, bukan per putaran loop. Inilah alasan mengapa pada output terlihat tulisan "Swap 1", "Swap 2", dan seterusnya.

Pada Bagian Insertion Sort:

- `print(f"Step {i}: {arr}")`: Perintah cetak diletakkan setelah satu angka selesai "disisipkan" ke posisi yang benar di bagian kiri array yang sudah terurut.